

Теорія інформації та кодування

Лекція 8. Системи стиснення інформації. Статистичні методи стиснення

Мета стискання (або компресії) даних – забезпечити компактне подання даних, створених джерелом, для їхнього економнішого зберігання та передавання через канали зв'язку.



Кодер джерела перетворює дані до стисненого вигляду, а декодер їх відновлює. Відновлені дані можуть або точно збігатися з початковими, або трохи від них відрізнятися.

Системи стиснення інформації поділяють за їхнім:

функціональним призначенням;

ступенем і швидкістю перетворення даних;

ступенем їхнього відновлення.

За функціональним призначенням розрізняють системи стиснення для передавання даних і для їхнього архівування. Різниця полягає в тому, що перші діють з незначними за обсягами даними (до декількох десятків і сотень байтів), а інші – з набагато більшим обсягом інформації (мегабайти).

За характером стиснення даних виділяють лінійні, матричні, комбіновані й каскадні способи стиснення:

до лінійних належать такі способи стиснення, за якими кодування виконується в одному напрямі (горизонтальному або вертикальному);

до матричних належать способи стиснення, за якими використовують матричний принцип заміни повторюваних елементів інформаційного масиву;

до комбінованих належать системи стиснення, у яких застосовують два або більше лінійних або матричних способів стиснення;

до каскадних належать системи послідовного стиснення інформації різними способами.

За ступенем стиснення розрізняють:

низькоефективні (з коефіцієнтом стиснення $r < 1.5$);

середньоефективні ($r = 1.5 - 3$);

високоефективні $r > 3$.

За швидкістю перетворення інформації системи стиснення поділяють на низько-, середньо- і високошвидкісні, для яких швидкість R змінюється від кількох кілобайтів до мегабайтів за секунду.

Всі алгоритми стиснення ґрунтуються на трьох теоретичних способах зменшення надлишковості даних.

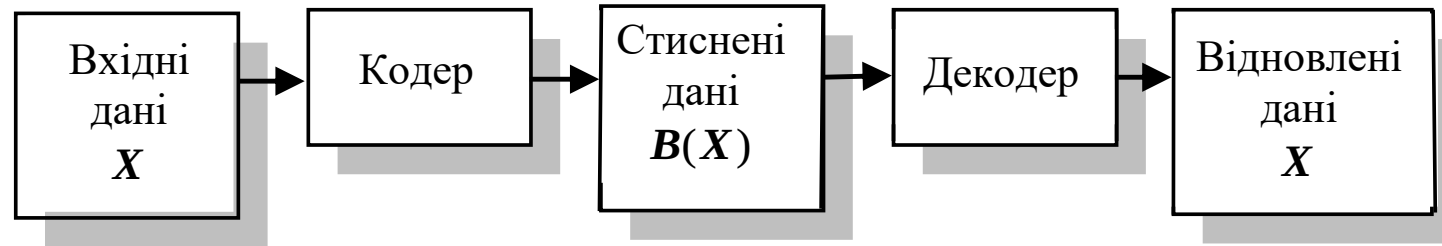
Перший спосіб полягає в зміні структури даних, другий – у зміні вмісту даних, а третій – в одночасній зміні як структури, так і вмісту даних.

Якщо під час стиснення даних відбувається тільки зміна структури даних, то метод стиснення є зворотним. У цьому випадку з архіву можна відновити інформацію повністю.

Якщо ж у ході стиснення даних відбувається зміна їхнього вмісту, то метод стиснення є незворотним, тобто в разі відновлення (розархівування) даних з архіву не відбувається повного відновлення інформації. Такі методи часто називають методами стиснення з регульованими втратами інформації.

За ступенем відновлення інформації розрізняють системи стиснення без втрат інформації (неруйнівне стиснення) та системи стиснення із втратами (руйнівне стиснення).

У системах стиснення без втрат декодер відновлює дані джерела абсолютно точно.



Вектор даних, що підлягає стисненню, є послідовністю $\mathbf{X} = (x_1, x_2, \dots, x_n)$ скінченної довжини. Елементарні повідомлення, що утворюють вектор $\mathbf{X}$ формують зі скінченного алфавіту джерела.

Виходом кодера є стиснені дані у вигляді двійкової послідовності $\mathbf{B}(\mathbf{X}) = (b_1, b_2, \dots, b_k)$ довжина якої k залежить від $\mathbf{X}$. Оскільки система стиснення неруйнівна, то однаковим векторам $\mathbf{B}(\mathbf{X}_l) = \mathbf{B}(\mathbf{X}_m)$ відповідають однакові кодові слова на вході кодера $\mathbf{X}_l$ та $\mathbf{X}_m$.

Коефіцієнт стиснення r — це відношення розміру початкових даних до розміру стиснених даних

$$r = \frac{n \log_2(\dim X)}{k}$$

де $\dim(X)$ — обсяг алфавіту джерела; n — довжина початкового повідомлення; k — довжина стисненого повідомлення.

Швидкість стиснення R визначається співвідношенням

$$R = \frac{k}{n}$$

Швидкість стиснення відображає, скільки бітів коду припадає на одне елементарне повідомлення джерела, тобто є середньою довжиною коду для повідомлення завдовжки n .

Чотири основні теоретичні алгоритми:

алгоритм RLE (Run Length Encoding);

алгоритми групи KWE(KeyWord Encoding);

алгоритм Хафмана;

арифметичний алгоритм.

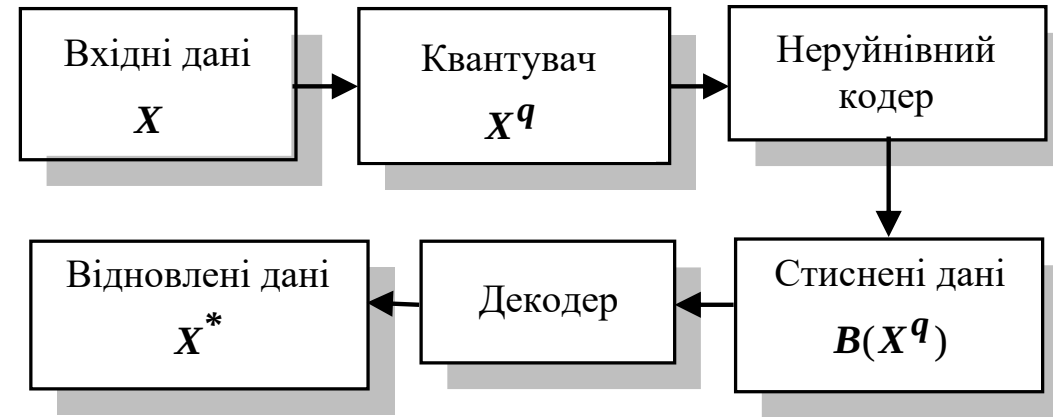
Приклади форматів стиснення без втрати інформації:

GIF (Graphics Interchange Format), TIFF (Tagged Image File Format) для графічних даних;

AVI для відеоданих;

ZIP, ARJ, RAR, CAB для довільних типів даних.

У системах стиснення з втратами інформації кодування відбувається так, що декодер не в змозі відновити дані джерела в первинному вигляді.



На відміну від кодування без втрат, у цій схемі наявний квантувач, що перетворює вектори вхідних даних X на вектор X^q достатньо близький до X за значенням середньоквадратичної похибки.

Далі кодер піддає неруйнівному стисненню вектор квантованих даних X^q так, щоб була забезпечена однозначна відповідність між X^q та $B(X^q)$, тобто для будь-яких $B(X_l^q) = B(X_m^q)$ виконується умова $X_l^q = X_m^q$, проте $X \neq X^*$

Руйнівний кодер характеризують швидкістю стиснення R та величиною спотворень D , які обчислюють за формулами

$$R = \frac{k}{n}, \quad D = \frac{1}{n} \sum \left(x_i - x_i^* \right)^2$$

Параметр R характеризує швидкість стиснення у бітах на одне елементарне повідомлення, а параметр D є мірою середньоквадратичної різниці між $\mathbf{X}$ та $\mathbf{X}^*$.

Прикладами форматів стиснення з втратами інформації можуть бути:

JPEG (Joint Photographic Experts Group) для графічних даних;

MPG для відеоданих;

MP3 для аудіоданих.

Канонічний алгоритм Хафмена

Під час стискання даних алгоритмом Хафмена необхідно два читання вхідного масиву даних – один для збирання статистичних даних та сортування символів за спаданням частот їхньої появи в масиві, а другий власне для кодування.

На першому проході:

- 1) кодер проглядає послідовність і визначає частоти появи F_i кожного символа в послідовності $\mathbf{X} = (x_1, x_2, \dots, x_n)$ довжини n ;
- 2) складає список символів алфавіту $\mathbf{A} = (a_1, a_2, \dots, a_k)$ у порядку спадання їхніх частот $\mathbf{F} = (F_1, F_2, \dots, F_k)$ появи в масиві даних $\mathbf{X} = (x_1, x_2, \dots, x_n)$;
- 3) кожному символу a_i призначає кодове слово $Code(a_i)$ за алгоритмом Хафмена довжиною l_i .

На другому проході:

- 1) побудований код використовує для кодування послідовності повідомлення;
- 2) виходом кодера є кодове слово, яке складається з двох частин

$$Code(\mathbf{X}) = Code_1(\mathbf{X}) + Code_2(\mathbf{X})$$

Перша частина, $Code_1(\mathbf{X})$ містить інформацію про використаний (застосований) код, друга $Code_2(\mathbf{X})$ – власне закодовану (стиснену) послідовність.

Тоді довжина кодової послідовності на виході кодера джерела інформації

$$l(Code(\mathbf{X})) = l(Code_1(\mathbf{X})) + l(Code_2(\mathbf{X}))$$

причому

$$l(Code_2(\mathbf{X})) = l_1F_1 + l_2F_2 + \dots + l_kF_k$$

Коефіцієнт стиснення

$$r = \frac{n \cdot b_n}{l(\text{Code}(X))}$$

Декодер спочатку за $\text{Code}_1(\mathbf{X})$ будує код, а потім за $\text{Code}_2(\mathbf{X})$ відновлює один за одним закодовані символи повідомлення.

Приклад. Нехай джерелом інформації є дані, які зберігаються в пам'яті комп'ютера у вигляді байтів, тобто обсяг алфавіту $k = 256$ ($b_n = 8$). Як тестову послідовність закодуємо прислів'я

КОХАННЯ_ВБИВАЄ_ЧАС,_А_ЧАС_ВБИВАЄ_КОХАННЯ

Позначимо $l_1 = \text{Code}_1(\mathbf{X})$, а $l_2 = \text{Code}_2(\mathbf{X})$.

Символ x_i	Частота F_i	p_i	Код $Code(x_i)$	l_i	$p_i l_i$
<i>A</i>	7	0,175	0 0	2	0,35
' _ '	6	0,15	0 1	2	0,3
<i>B</i>	4	0,1	1 1 0	3	0,3
<i>H</i>	4	0,1	1 1 1	3	0,3
<i>K</i>	2	0,05	1 0 1 0	4	0,2
<i>O</i>	2	0,05	1 0 1 1	4	0,2
<i>Б</i>	2	0,05	1 0 0 0 0 0	6	0,3
<i>Є</i>	2	0,05	1 0 0 0 0 1	6	0,3
<i>И</i>	2	0,05	1 0 0 0 1 0	6	0,3
<i>С</i>	2	0,05	1 0 0 0 1 1	6	0,3
<i>X</i>	2	0,05	1 0 0 1 0 0	6	0,3
<i>Ч</i>	2	0,05	1 0 0 1 0 1	6	0,3
<i>Я</i>	2	0,05	1 0 0 1 1 0	6	0,3
' , '	1	0,025	1 0 0 1 1 1	6	0,15
	40	$\sum p_i = 1$	$\bar{l} = \sum p_i l_i = 3.9$		

Маємо

$$l_2 = 14 + 10 \cdot 12 + 8 \cdot 2 + 6 = 156 \text{ біт}$$

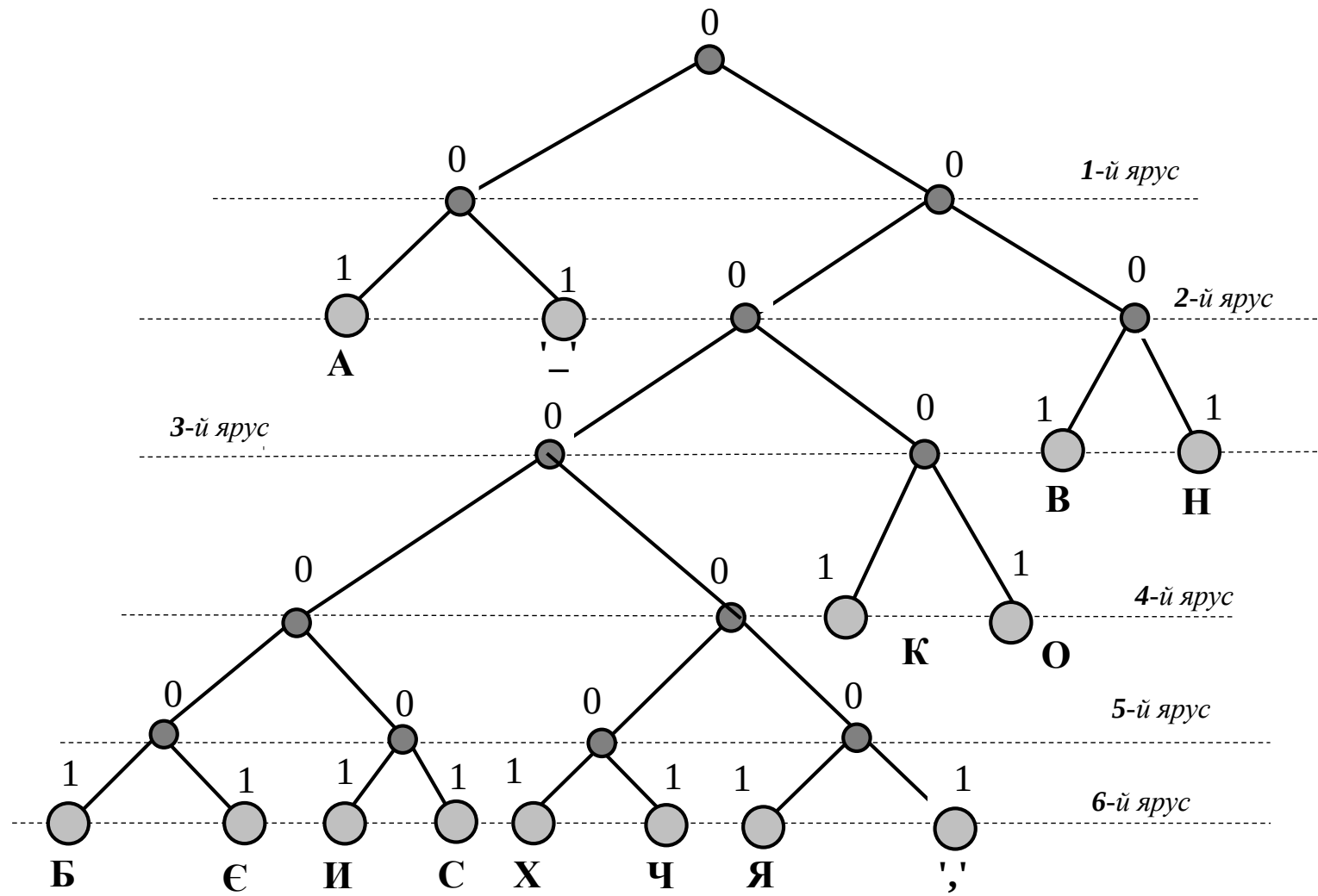
Щоб обчислити l_1 потрібно домовитися про спосіб передавання інформації про код.

Передаватимемо структуру кодового дерева. Знаючи цю структуру, декодер сам однозначно відновить усі кодові слова.

Усім вершинам дерева присвоєно двійкові символи (листкам — 1, а вузлам — 0). Рахуватимемо символи, присвоєні вершинам, ярус за ярусом, починаючи з кореня дерева (йому також присвоєно символ 0).

Отримаємо:

000 1100 0011 0011 0000 11111111



Для остаточного задання опису коду, необхідно закодувати кожен символ (з таблиці ASCII+). Для цього достатньо 8 біт на кожний символ.

У підсумку довжина службової інформації становить

$$l_1 = 27 + 8 \cdot 14 = 139 \text{ біт}$$

Довжина всього стисненого повідомлення

$$l = l_1 + l_2 = 139 + 156 = 295 \text{ біт}$$

Без стискання на кодування повідомлення потрібно було б витратити

$$8 \cdot 40 = 320 \text{ біт}$$

Лема . Повне двійкове дерево, яке має k листків, має $k - 1$ вузол. Для повного опису дерева достатньо $2k - 1$ біт.

Доведення. Метод математичної індукції за ярусами.

$n = 1$, $k = 1$ листок, $k - 1 = 0$ вузлів.

$n = 2$, $k = 2$ листки, $k - 1 = 1$ вузол.

Нехай при $n = t$ твердження виконується. Тобто, якщо у дереві було t ярусів, то при кількості листків k було $k - 1$ вузлів.

Нехай $n = t + 1$. Маємо, з припущення індукції, що в цьому дереві є $2k$ листків, а кількість вузлів дорівнює сумі кількостей листків і вузлів з дерева розглянутого в припущенні індукції, тобто $2k - 1$, лему доведено.

Теорема. У разі двопрхідного кодування послдовності $\mathbf{X} = (x_1, x_2, \dots, x_n)$ з використанням коду Хафмена для дискретного джерела з обсягом алфавіту k та ентропією H середня швидкість кодування (стискання) задовольняє нерівність

$$\bar{R} \leq H + 1 + \frac{1}{n} (k \lceil \log_2 k \rceil + 2k - 1)$$

Доведення. Довжина кодового слова для повідомлення $\mathbf{X} = (x_1, x_2, \dots, x_n)$ дорівнює

$$l(\text{Code}(\mathbf{X})) = l(\text{Code}_1(\mathbf{X})) + l(\text{Code}_2(\mathbf{X})) = l_1 + l_2$$

$$l_1 = \text{Code}_1(\mathbf{X}), \text{ а } l_2 = \text{Code}_2(\mathbf{X})$$

Довжину закодованої службової інформації (інформації про код) l_1 можна розбити на дві частини: інформацію про кодове дерево та інформацію про те, який із символів відповідає кожному листку кодового дерева.

Верхню межу для довжини кодового слова, що описує двійкове дерево, обчислюють на підставі доведеної раніше леми, вона становить $2k - 1$.

Задання символу для одного листка потребує не більше $\lceil \log_2 k \rceil$ біт. Тому

$$l_1 \leq 2k - 1 + k \lceil \log_2 k \rceil$$

Далі

$$l_2 = \sum_{i=1}^k F_i l_i = n \sum_{i=1}^k \frac{F_i}{n} l_i = n \sum_{i=1}^k \frac{F_i}{n} \left\lceil \log_2 \frac{n}{F_i} \right\rceil \leq n(H + 1)$$

Нарешті

$$\bar{R} = \frac{l}{n} = \frac{l_1 + l_2}{n} \leq \frac{1}{n} (k \lceil \log_2 k \rceil + 2k - 1) + H + 1$$

З доведеної нерівності випливає, що ефективність кодування (стиснення) інформації можна оцінити за допомогою нерівності

$$\alpha = \bar{R} - H \leq 1 + \frac{C}{n}$$

де перший доданок у правій частині нерівності зумовлений надлишковістю використаного коду, другий – відсутністю інформації про статистику джерела, а C – константа, яка не залежить від n .

При $n \rightarrow \infty$ ефективність двопрхідного кодування прямує до ефективності кодування кодом Хафмена з відомим розподілом імовірностей появи символів.

Недоліки

1. Необхідність побудови таблиці ймовірностей (кодів) для кожного типу даних, які стискають. Тому у випадку, коли ймовірність символів невідома, ці алгоритми реалізують у два читання: у першому відбувається збирання частот символів, оцінка їхніх ймовірностей і складання таблиці кодів, друге використовують саме для стиснення.
2. Необхідність зберігання (передавання) таблиці кодів разом зі стисненим повідомленням, що знижує ефект стиснення.
3. Мінімальна довжина кодових слів не може бути меншою від одиниці, тоді як ентропія повідомлень може бути близькою до нуля. У цьому випадку оптимальні методи кодування виявляються суттєво надмірними.
4. Найліпший ефект стиснення досягають тільки у випадку, коли ймовірності символів є цілими від'ємними степенями двійки, тобто $1/2$, $1/4$, $1/8$ і т. д.

Характеристики:

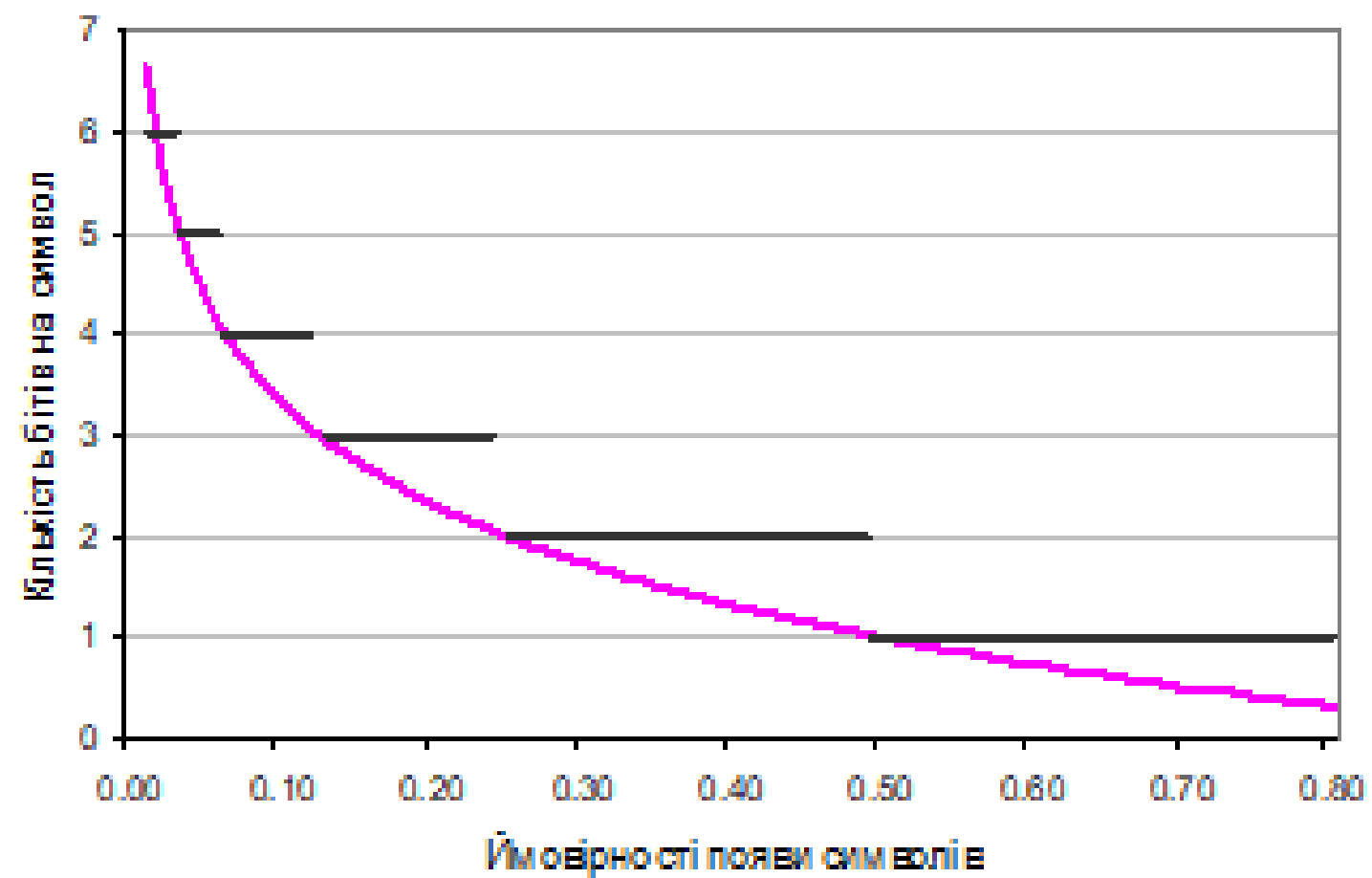
Коефіцієнт стискання: 8, 1,5, 1 (найліпший, середній, найгірший).

Симетричність за часом: 2:1 (завдяки тому, що потребує двох читань масиву даних, які підлягають стисканню).

Характерні особливості: один з небагатьох алгоритмів, що не збільшує розміру стиснутого файлу в найгіршому випадку (якщо не брати до уваги таблицю кодів, що записується на початок файлу).

Кодування з фіксованою таблицею застосовують на останньому етапі стискання в алгоритмі JPEG, в алгоритмі CCIT Group, а також у разі стискання звичайного тексту, наприклад, англійського з відомою кодовою таблицею.

Недоліки 3 та 4 долають використанням алгоритму до блоків символів, які отримують розбиттям усього повідомлення на окремі блоки, розглядаючи їх як одиниці повідомлення.



Арифметичне кодування стискає поточний файл, присвоюючи код усьому, зазвичай, великому файлу замість кодування окремих символів.

На першому етапі потрібно знайти частоти (ймовірності) появи символів у повідомленні (файлі), яке стискають. Найліпший результат отримуємо, прочитавши весь вхідний файл на першому проходженні алгоритму стискання.

За розподілом імовірностей появи символів у вхідному файлі складають таблицю інтервалів, де кожному символу ставиться у відповідність підінтервал з інтервалу $[0,1)$, довжина якого є пропорційною до ймовірності появи символа.

Об'єднання цих інтервалів утворює інтервал $[0,1)$, причому інтервали не перетинаються між собою ніде, окрім як у граничних точках.

Розглянемо роботу алгоритму на прикладі.

Нехай повідомлення (вхідний файл) складається з чотирьох символів $S_2S_1S_2S_3$.

Алфавіт цього повідомлення складається з трьох символів S_1, S_2, S_3 з імовірностями їхньої появи $p_1 = 0.25, p_2 = 0.5, p_3 = 0.25$.

Символ	Імовірність	Інтервал
S_1	0,25	$[0; 0,25)$
S_2	0,50	$[0,25 ; 0,75)$
S_3	0,25	$[0,75 ; 1)$

Щоб закодувати послідовність:

- 1) починаємо з інтервалу $[0, 1)$;
- 2) перший символ звужує цей інтервал до інтервалу $[0.25, 0.75)$, тобто відкидає від початкового інтервалу по 25 % з початку і з кінця;
- 3) наступний символ (S_1) звужує інтервал $[0.25, 0.75)$ до інтервалу $[0.25, 0.375)$, тобто відкидає від початкового інтервалу 75 % з кінця;
- 4) наступний символ (S_2) звужує інтервал $[0.25, 0.375)$ до інтервалу $[0.28125, 0.34375)$, тобто відкидає від початкового інтервалу по 25 % з початку і з кінця;
- 5) нарешті останній символ (S_3) звужує інтервал $[0.328125, 0.34375)$ до інтервалу $[0.28125, 0.34375)$, тобто відкидає від початкового інтервалу 75 % з початку;
- 6) остаточний результат кодування повідомлення це дійсне число, що належить інтервалу $[0.328125, 0.34375)$, яке можна отримати як частку від ділення цілого числа на мінімальний степінь 2

Символ	Інтервал	Інтервал кодування	Ширина інтервалу
S_2	$[0,25 ; 0,75)$	$[0,25 ; 0,75)$	0,5
S_1	$[0;0,25)$	$[0,25 ; 0,375)$	0,125
S_2	$[0,25 ; 0,75)$	$[0,28125 ; 0,34375)$	0,0625
S_3	$[0,75 ; 1)$	$[0,328125 ; 0,34375)$	0,015625

Ми отримуємо:

$$0.328125 = 21 / 2^6$$

Степінь 2 визначає розрядність коду повідомлення, а самим кодом буде двійкове представлення чисельника довжиною, що дорівнює степені 2 у знаменнику, тобто

$$Code(S_2S_1S_2S_3) = 21_{(10)} = 010101_{(2)}.$$

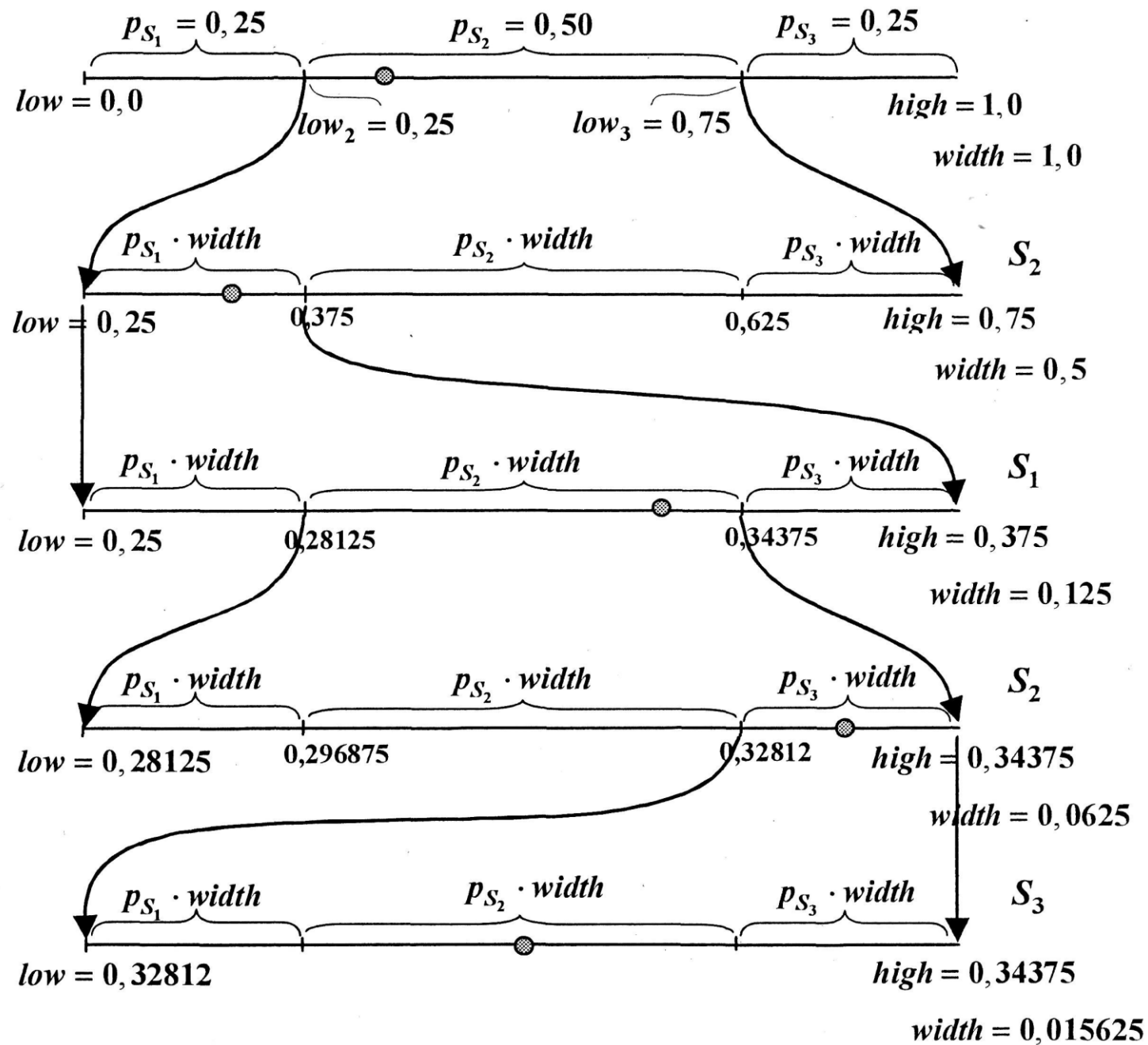
Підсумуємо (Алгоритм А1):

Крок 1. Задати поточний інтервал [$low = 0, high = 1$), його ширину $width = 1$, повідомлення $X = \{x_1, x_2, \dots, x_n\}$, його довжину n , таблицю інтервалів (для символу $x_i - [low_i, high_i)$, $i = 1, \dots, k$).

Крок 2. Для кожного символу S_i вхідного файлу вибрати підінтервал, що відповідає цьому символу і вважати його новим поточним інтервалом. Верхню і нижню границі нового інтервалу знаходять додаванням до початку попереднього інтервалу добутків його ширини на відповідні границі відрізка, що відповідає поточному символу повідомлення з таблиці інтервалів.

$$\begin{aligned} low &= low + width \cdot low_i \\ high &= low + width \cdot high_i \\ width &= high - low \end{aligned}$$

Крок 3. Коли весь вхідний файл буде прочитаний, виходом алгоритму є число, що належить остаточному інтервалу. Це число знаходять як частку від ділення цілого числа на мінімальний степінь двійки. Двійковий код цього цілого числа і буде арифметичним кодом повідомлення (довжина якого дорівнює відповідному степені 2).



Маємо:

ширина поточного *width* інтервалу на кожному кроці дорівнює добутку імовірностей символів до поточного включно, тобто $w = p(x_i)w$.

нижня границя інтервалу *low* на кожному кроці дорівнює сумі поточної нижньої границі та добутку імовірностей попередніх символів і кумулятивної імовірності поточного символу, тобто $l = l + q(x_i)w$, $q(x_i) = q_1 + \sum_{j=1}^{i-1} p(x_j)$, $q_1 = 0$, $j = 1, 2, \dots, k$.

на останньому кроці, довжина інтервалу дорівнює імовірності усього повідомлення.

Застосуємо аналог коду Гільберта-Мура і отримаємо алгоритм A2.

Алгоритм А2:

Крок 1. Задати поточний інтервал [$l = 0, h = 1$), повідомлення $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$, його довжину n , імовірності символів (для символу $x_i - p_i, i = 1, \dots, k$).

Крок 2. Обчислити кумулятивні імовірності символів.

$$\begin{aligned} q_1 &= 0; \\ \text{for } i &= 1 \text{ to } k \text{ do} \\ q_i &= q_{i-1} + p_{i-1} \end{aligned}$$

Крок 3. Кодування (i – номер символу у алфавіті)

$$\begin{aligned} w &= h \\ \text{for } j &= 1 \text{ to } n \text{ do} \\ l &= l + q_i w \\ w &= p_i w \end{aligned}$$

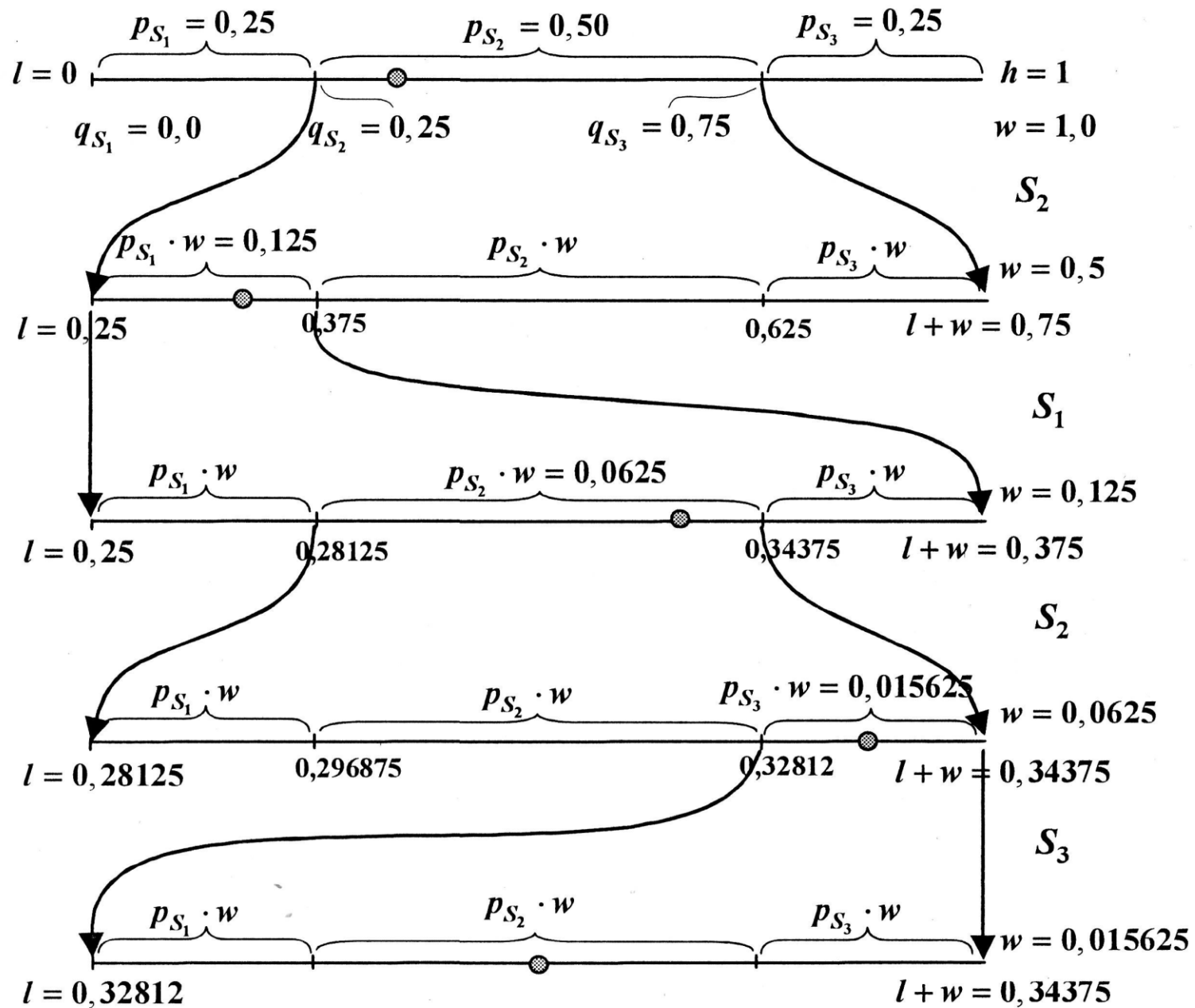
Крок 4. Формування кодового слова $\text{Code}(\mathbf{X})$ перші $\lceil -\log_2 w / 2 \rceil$ розрядів після коми у двійковому записі числа $l + w / 2$.

Нехай повідомлення (вхідний файл) складається з чотирьох символів $S_2S_1S_2S_3$.

Маємо

Символ x_i	Імовірність $p(x_i)$	Кумулятивна імовірність $q(x_i)$
S_1	0.25	0
S_2	0.5	0.25
S_3	0.25	0.75

Символ x_i	Імовірність $p(x_i)$	Кумулятивна імовірність $q(x_i)$	Нижня границя інтервалу l	Ширина інтервалу w
		0	0	1
S_2	0.5	0.25	0.25	0.5
S_1	0.25	0	0.25	0.125
S_2	0.5	0.25	0.28125	0.0625
S_3	0.25	0.75	0.328125	0.015625
Довжина кодового слова $\lceil -\log_2 w/2 \rceil = 7$ $l + w/2 = 0.3359375_{(10)}$ $Code(S_2S_1S_2S_3) = 0101011$				



Важливою особливістю арифметичного кодування є його здатність сильно стискати довгі послідовності однакових символів.

Приклад. Припустимо, що потрібно закодувати такий рядок символів: SSSSSSSSS&.

Маємо імовірність появи S становить 0.9, а & - 0.1. Отже

Символ	Інтервал кодування	Ширина інтервалу
S	[0,0 ; 0,9)	0,9
S	[0,0 ; 0,81)	0,81
S	[0,0 ; 0,729)	0,729
S	[0,0 ; 0,6561)	0,6561
S	[0,0 ; 0,59049)	0,59049
S	[0,0 ; 0,531441)	0,531441
S	[0,0 ; 0,4782969)	0,4782969
S	[0,0 ; 0,43046721)	0,43046721
S	[0,0 ; 0,387420489)	0,387420489
&	[0,348678440 ; 0,387420489)	0,038742049

Отримаємо, що результатом кодування є число з інтервалу $[0.34867844, 0.387420489)$, яке можна подати як частку від ділення цілого числа на мінімальний степінь двійки, а саме $0.375 = 3 / 2^3$.

Тому

$$Code(SSSSSSSSS\&) = 011.$$

Коефіцієнт стискання

$$r = 80 / 3 \approx 26.7$$

Декодування арифметичного коду.

Почнемо знову з прикладу. Нехай $Code(X) = 010101$ та ми маємо таблицю

Символ	Імовірність	Інтервал
S_1	0,25	$[0; 0,25)$
S_2	0,50	$[0,25 ; 0,75)$
S_3	0,25	$[0,75 ; 1)$

Маємо $010101_{(2)} = 21_{(10)}$ звідки отримуємо число $21/2^6 = 0.328125$. Це число є значенням поточного коду. З таблиці інтервалів отримуємо, що першим символом повідомлення є S_2 , оскільки 0.328125 належить інтервалу $[0.25, 0.75)$.

Вилучимо з результату декодування вплив тепер уже відомого символу S_2 . Для цього віднімемо від поточного коду нижню границю інтервалу розкодованого символу і розділимо отриманий результат на ширину цього інтервалу, тобто наступне значення поточного коду буде

$$(0.328125 - 0.25) / 0.5 = 0.15625.$$

Оскільки число 0.15625 належить інтервалу $[0, 0.25)$, то наступним символом у повідомленні є S_1 .

Знову вилучимо з результату декодування вплив тепер уже відомого символу S_1 , отримаємо

$$(0.15625 - 0) / 0.25 = 0.625$$

Оскільки 0.625 належить інтервалу $[0.25, 0.75)$, то третій символ повідомлення S_2 . Вилучимо з результату декодування вплив цього символу, отримаємо

$$(0.625 - 0.25) / 0.5 = 0.75$$

Отже, четвертим символом повідомлення є S_3 , оскільки 0.75 належить інтервалу $[0.75, 1)$.

Оскільки число 0.15625 належить інтервалу $[0, 0.25)$, то наступним символом у повідомленні є S_1 .

Знову вилучимо з результату декодування вплив тепер уже відомого символу S_1 , отримаємо

$$(0.15625 - 0) / 0.25 = 0.625$$

Оскільки 0.625 належить інтервалу $[0.25, 0.75)$, то третій символ повідомлення S_2 . Вилучимо з результату декодування вплив цього символу, отримаємо

$$(0.625 - 0.25) / 0.5 = 0.75$$

Отже, четвертим символом повідомлення є S_2 , оскільки 0.75 належить інтервалу $[0.75, 1)$.

Отже, підсумуємо декодування алгоритму A1:

Крок 1. Ініціалізувати таблицю інтервалів символів алфавіту.

Крок 2. Ввести довжину декодованої послідовності n та код повідомлення $Code(X)$ і взяти це число як значення поточного коду

$$j = 1$$

Крок 3. За таблицею інтервалів символів алфавіту визначають відрізок, що містить значення поточного коду, і за цим інтервалом з тієї самої таблиці однозначно визначають символ повідомлення, тобто

for $i = 1$ to k do
if $((Code(X) < high_i) \text{ and } (Code(X) \geq low_i))$ then $x_j = S_i$
go to Крок 4

Крок 4. Від поточного коду віднімають нижню границю його інтервалу. Різницю ділять на довжину цього інтервалу. Отримане значення вважають новим значенням поточного коду. Перехід до кроку 3, тобто

$$Code(X) = (Code(X) - low_i) / width_i$$

if ($j = n$) go to Вихід
 $j = j + 1$
go to Крок 3

Вихід. Декодована послідовність $X = \{x_1, x_2, \dots, x_n\}$

Декодування алгоритму A2:

Крок 1. Ініціалізувати таблицю з кумулятивним імовірностями символів алфавіту.

Крок 2. Ввести довжину декодованої послідовності n та код повідомлення $Code(\mathbf{X})$ і взяти це число як значення поточного коду

$$l = 0$$

$$w = 1$$

$$q_{k+1} = 1$$

Крок 3. Декодування

for $j = 1$ to n do

$$i = 1$$

while $(l + q_i w) < Code(\mathbf{X})$ do $i = i + 1$

$$l = l + q_{i-1} w$$

$$w = p_{i-1} w$$

$$x_j = S_{i-1}$$

Вихід. Декодована послідовність $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$

l	w	Гіпотеза щодо x_i	$q(x)$	$l+qw$	Символ на виході
0101011=Code(x)=0,3359375					
0	1	S_1	0,0	$0,000000 < Code(x)$	S_2
		S_2	0,25	$0,250000 < Code(x)$	
		S_3	0,75	$0,750000 > Code(x)$	
0,25	0,5	S_1	0,0	$0,250000 < Code(x)$	S_1
		S_2	0,25	$0,375000 > Code(x)$	
0,25	0,125	S_1	0,0	$0,250000 < Code(x)$	S_2
		S_2	0,25	$0,281250 < Code(x)$	
		S_3	0,75	$0,343750 > Code(x)$	
0,28125	0,0625	S_1	0,0	$0,281250 < Code(x)$	S_3
		S_2	0,25	$0,296875 < Code(x)$	
		S_3	0,75	$0,328125 < Code(x)$	

Перша проблема полягає в тому, що декодеру потрібно знати про закінчення процедури декодування.

Друга проблема випливає з самої ідеї арифметичного кодування і полягає в тому, що кінцевий результат кодування – скінченний інтервал – стане відомим тільки після закінчення процесу кодування і, як наслідок, потрібно затримати передавання (записування) кодового слова, доки не буде закодований останній символ повідомлення.

Третя проблема – це питання точності.

Характеристики алгоритму арифметичного кодування .

Коефіцієнт стискання: >8 , 1 (ліпший, найгірший).

Плюси алгоритму: забезпечує ліпший ступінь стискання, ніж алгоритм Хафмена (на типових даних до 10 %).

Характерні особливості: як і кодування за Хафменом, не збільшує розміру стиснутого файлу в найгіршому випадку